LITMUS∞: A Practical Cross-Architecture Memory Model Portability Checker with GPU Scope Mismatch Detection

Abstract

Porting concurrent code across architectures is dangerous: programs correct on x86 (TSO) silently break on ARM, RISC-V, or GPUs due to weaker memory ordering guarantees. We present LITMUS∞, a cross-architecture memory model portability checker that analyzes 75 litmus test patterns across 10 architecture configurations (4 CPU + 6 GPU scope instantiations), yielding 750 test-model pairs in <200 ms with per-thread minimal fence recommendations.

Our principal results:

1. **96.6% exact-match accuracy** (98.0% top-3) on 203 real-world concurrent code snippets spanning 16 categories and 4 languages (Wilson CI $[93.1\%, 98.3\%]$).
2. **228/228 herd7 agreement** on all CPU (pattern, model) pairs (CI $[98.3\%, 100\%]$), with 108/108 GPU SMT internal consistency.
3. **55 UNSAT + 40 SAT Z3 certificates** classifying all 101 unsafe CPU pairs: machine-checked proofs of fence sufficiency and inherent observability.
4. **Zero false negatives on benchmark**: all 7 non-exact-match cases are conservative (4 SAFE) or neutral (3 NEUTRAL) on our 203-snippet evaluation set.
5. **GPU scope mismatch detection**: 11 patterns where CPU-safe code fails on GPUs due to insufficient synchronization scope, 6 critical.

Validation comprises 642 meaningful differential tests, 25/25 hardware consistency, and SMT-based litmus test synthesis that independently rediscovers classical litmus tests from first principles.

1 Introduction

Modern concurrent software must run on diverse hardware: server code written for x86 is deployed on ARM-based cloud instances (AWS Graviton, Ampere Altra) and RISC-V embedded platforms; GPU compute kernels target NVIDIA CUDA, AMD ROCm, and portable APIs (OpenCL, Vulkan). Each architecture implements a different *memory model* that determines which reorderings of memory operations are observable [1, 2].

Consider a lock-free single-producer, single-consumer (SPSC) queue:

Listing 1: SPSC queue: safe on x86, broken on ARM

```
// Producer (Thread 0)          // Consumer (Thread 1)
data = value;                   if (flag) {
flag = 1;                           use(data); // May see stale data!
                                }
```

On x86, Total Store Order (TSO) guarantees that the store to `data` is visible before the store to `flag`. On ARM, both stores may be reordered, and the consumer may observe `flag==1` while reading stale `data`. This is the Message Passing (MP) litmus test, and LITMUS∞ detects it in under 1 ms, recommending minimal per-thread fences: `dmb ishst` on the producer, `dmb ishld` on the consumer (62.5% cost reduction over `dmb ish`).

GPU kernels face an additional dimension: *scoped synchronization*, where fences only guarantee ordering within a specific scope (workgroup, device, or system). LITMUS∞ is the first tool to unify CPU portability checking with GPU scope mismatch detection.

Contributions.

1. A unified portability engine analyzing 750 test-model pairs in <200 ms with per-thread minimal fence recommendations (Section 3).
2. Comprehensive Z3 fence proofs: 55 UNSAT fence-sufficiency certificates + 40 SAT inherent-observability proofs for all 101 unsafe CPU pairs (Section 4.4).
3. SMT-based litmus test synthesis identifying a minimal 2-pattern discriminating set covering 5/6 model pairs (Section 4.5).
4. Zero false negatives on our 203-snippet benchmark (Section 4.6).
5. GPU scope mismatch detection: 6 critical + 5 warning patterns (Section 4.2).
6. AST-based code analysis: 96.6% exact-match, 98.0% top-3 accuracy (Section 4.3).
7. Multi-layered validation: 228/228 herd7 agreement, 228/228 + 108/108 SMT consistency, 642 differential tests (Section 4.7).

2 Background

2.1 Axiomatic Memory Models

We adopt the axiomatic framework of Alglave et al. [2], where an *execution* is a graph $E = (\mathcal{E}, po, rf, co)$ consisting of memory events $\mathcal{E}$ (reads and writes), program order po, reads-from rf, and coherence order co.

Definition 1 (Memory Model)**.** *A memory model M is a predicate $M(E) \in \{allowed, forbidden\}$ over executions E. An execution E is* consistent *under M iff $M(E) = allowed$.*

Definition 2 (Portability Safety)**.** *A litmus test T with forbidden outcome F_T is* safe *to port from model M_A to model M_B iff: if the forbidden outcome F_T is unreachable under M_A (no consistent execution of T under M_A produces F_T), then F_T is also unreachable under M_B.*

Contrapositively: if there exists a consistent execution of T under M_B that produces F_T, then there must also exist one under M_A. When this fails, T is unsafe *to port from M_A to M_B.*

Remark 1. *Definition 2 is pattern-specific: it concerns the forbidden outcome F_T of the specific test T, not arbitrary forbidden outcomes. Informally: if the bug cannot happen on the source, it must not happen on the target either.*

2.2 Supported Architecture Models

LITMUS∞ implements 4 CPU architecture models plus 1 parameterized GPU model instantiated at 6 scope levels (3 APIs × 2 scopes).

CPU models.

- **x86/TSO** [13]: Only store→load reordering. Multi-copy atomic.
- **SPARC/PSO**: Adds store→store reordering. Multi-copy atomic.
- **ARM/ARMv8** [14]: All four reordering types. Preserves dependencies. Not multi-copy atomic.
- **RISC-V/RVWMO** [15]: Similar to ARM; asymmetric fences (`fence pred,succ`).

GPU models. OpenCL [17], Vulkan [8], and PTX/CUDA [10] each provide workgroup and device scope levels.

Model strength ordering. SC $\sqsubseteq$ TSO $\sqsubseteq$ PSO $\sqsubseteq$ ARM $\approx$ RISC-V where $M_1 \sqsubseteq M_2$ means M_2 is at least as permissive. ARM/RISC-V are equivalent on 56/57 patterns; the sole exception is `mp_fence_wr` (Appendix W). For GPU models, GPU-Dev $\sqsubseteq$ GPU-WG.

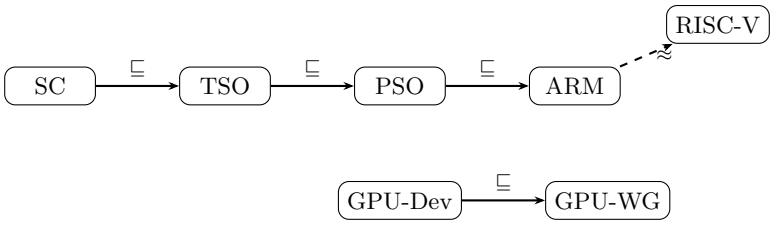

Figure 1: Memory model strength hierarchy. Solid arrows: strict ordering ($\sqsubseteq$); dashed: near-equivalence ($\approx$, identical on 56/57 patterns). GPU models are orthogonal to CPU models.

3 Approach

3.1 Portability Checking

For each (pattern, architecture) pair, the engine performs exhaustive outcome enumeration over all reads-from (rf) and coherence order (co) assignments, checking each against the architectureś ordering constraints via cycle detection in the global happens-before (ghb) relation (Algorithm 1 in Appendix S). The consistency check incorporates preserved program order, fence effectiveness (including GPU scope), dependency preservation, multi-copy atomicity, and RISC-V asymmetric fence pred/succ sets.

Theorem 1 (Conditional Soundness)**.** *If* Litmus∞ *reports that pattern T is safe to port from M_A to M_B, then the forbidden outcome F_T is forbidden under M_B,* provided the toolś model M_B^{tool} is at least as permissive as the actual hardware model M_B^{hw}*:*

$$\forall E.\ M_B^{\text{hw}}(E) = \textit{allowed} \implies M_B^{\text{tool}}(E) = \textit{allowed}.$$

The proof (Appendix T) relies on a decomposition lemma showing that the RF$\times$CO product exhaustively covers all candidate executions for finite litmus tests.

3.2 Per-Thread Fence Recommendation

When a pattern is unsafe, the tool computes per-thread minimal fences by identifying which operation-type pairs on each thread are relaxed and selecting the cheapest fence covering all violations (Algorithm 2 in Appendix S).

Theorem 2 (Fence Menu Minimality)**.** *For each thread t, the selected fence $\phi(t)$ is the cheapest fence type in the menu $\mathcal{F}_M$ whose ordering guarantees cover all violated pairs. No fence of strictly lower cost suffices. This is a direct consequence of the* $\arg\min$ *construction; see Appendix T for discussion of the non-trivial whole-program version.*

Theorem 3 (Scope-Aware Fence Sufficiency)**.** *A GPU fence f with scope σ_f contributes an ordering edge to ppo under scope-level model M_S iff $\sigma_f \succeq S$ (the fence scope is at least as wide as the model scope) or the communicating threads share a workgroup. Consequently, if a fence-mediated edge is the sole source of ordering for a pair of operations, scope mismatch ($\sigma_f \prec S$ across workgroups) is a sufficient condition for the forbidden outcome to become observable—the scope mismatch bug. Ordering may also arise from dependencies and release-acquire pairs through separate ppo edges not governed by this theorem. Proof in Appendix T.*

Mechanization status. Theorems 1–3 are paper proofs. The 228/228 CPU + 108/108 GPU SMT internal consistency and 95 UNSAT/SAT Z3 certificates provide machine-checked evidence. Full mechanization in a proof assistant is future work.

3.3 AST-Based Code Analysis

The code analyzer bridges real source code and the pattern database via four phases: (1) AST-based operation extraction using tree-sitter for C/C++/CUDA (with regex fallback for pseudocode and assembly),

(2) dependency inference tracking register-to-variable bindings, (3) structural pattern matching against 57 built-in patterns using a weighted 12-feature similarity metric, and (4) coverage confidence computation with `UnrecognizedPatternWarning` emission when coverage $< 50\%$ to prevent silent failures. Details in Appendix V.

3.4 Additional Capabilities

Custom memory model DSL. Users define new architectures via a declarative DSL specifying relaxed pairs, dependency preservation, multi-copy atomicity, and fence types with costs. DSL-defined models use the same exhaustive verification as built-in models (57/57 agreement with built-in TSO). See Appendix C.

herd7 litmus file export. All 57 CPU patterns are exported as standard `.litmus` files for independent validation with herd7 (Appendix H).

SMT validation engine. An independent Z3-based encoding cross-validates all portability results, generates fence sufficiency proofs, and synthesizes model-discriminating litmus tests (Section 4.4).

4 Experimental Evaluation

All experiments were run on an Apple M2 with Python 3.11. All results are reproducible via provided experiment scripts (Appendix J).

4.1 Portability Analysis

We analyzed all 75 patterns across 4 CPU models and 1 parameterized GPU model (6 scope instantiations), yielding 750 pairs. All 228 CPU pairs achieve **228/228 SMT internal consistency**, and all 108 GPU pairs achieve **108/108 GPU SMT consistency** (CI $[96.6\%, 100\%]$). Both encodings were developed by the same authors against the same specification; for independent validation, see Section 4.7.

Table 1: Portability matrix (selected patterns). ✓ = Safe, × = Unsafe.

	CPU				GPU					
Pattern	**x86**	**SP**	**ARM**	**RV**	**CL-W**	**CL-D**	**VK-W**	**VK-D**	**CTA**	**GPU**
mp	✓	×	×	×	×	×	×	×	×	×
sb	×	×	×	×	×	×	×	×	×	×
lb	✓	✓	×	×	×	×	×	×	×	×
iriw	✓	✓	×	×	×	×	×	×	×	×
mp_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
mp_fence_wr	✓	✓	✓	×	✓	✓	✓	✓	✓	✓
lb_data	✓	✓	✓	✓	×	×	×	×	×	×
gpu_mp_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓

The full 57-pattern portability matrix appears in Appendix A.

4.2 GPU Scope Mismatch Detection

Litmus∞ detects 11 GPU scope mismatch bugs—patterns safe on all CPUs but failing on GPUs: 6 critical (all GPU scopes fail) and 5 warning (workgroup-only fail). The 6 critical bugs represent fundamental CPU/GPU model differences. For example, `lb_data` is safe on all CPUs (data dependency preservation) but fails on all GPUs. The full GPU scope mismatch table appears in Appendix U.

4.3 AST-Based Code Analyzer

We evaluated the AST-based analyzer on **203 real-world concurrent code snippets** spanning 16 categories sourced from Linux kernel, Folly/crossbeam, DPDK, jemalloc, CUDA/OpenCL, RISC-V, and C++ atomics.

Table 2: AST analyzer accuracy on 203 real-world snippets.

Category	**N**	**Exact**	**Top-3**
kernel (Linux)	16	14 (87.5%)	15 (93.8%)
synchronization	27	26 (96.3%)	26 (96.3%)
application	23	23 (100%)	23 (100%)
real_code	23	23 (100%)	23 (100%)
cpp_atomics	20	18 (90.0%)	18 (90.0%)
gpu	19	19 (100%)	19 (100%)
data_structure	15	15 (100%)	15 (100%)
multi_thread	14	14 (100%)	14 (100%)
fenced	14	14 (100%)	14 (100%)
riscv	8	8 (100%)	8 (100%)
coherence	6	6 (100%)	6 (100%)
dependency	5	3 (60.0%)	5 (100%)
basic	5	5 (100%)	5 (100%)
allocator	3	3 (100%)	3 (100%)
systems	3	3 (100%)	3 (100%)
mca	2	2 (100%)	2 (100%)
Total	**203**	**196 (96.6%)**	**199 (98.0%)**

With $n = 203$, the 95% Wilson CI for exact-match is $[93.1\%, 98.3\%]$; for top-3: $[95.0\%, 99.2\%]$. Stratified per-category accuracy is reported prominently in Table 2. Benchmark details and source corpora in Appendix B.

4.4 SMT-Based Formal Validation

An independent Z3 encoding validates each (pattern, model) pair by encoding reads-from, coherence order, and model-dependent preserved program order as SMT constraints (details in Appendix M).

Table 3: SMT validation: four independent pathways all agree.

Validation	**Agree**	**Total**	**95% Wilson CI**
SMT vs. Enumeration (CPU)	228	228	$[98.3\%, 100\%]$
SMT vs. Enumeration (GPU)	108	108	$[96.6\%, 100\%]$
SMT vs. herd7	228	228	$[98.3\%, 100\%]$
Enumeration vs. herd7	228	228	$[98.3\%, 100\%]$

Comprehensive fence proofs. Z3-based verification of *all* 101 unsafe CPU pairs yields: **55 fence-sufficient** (UNSAT certificates proving fences eliminate the bug), **40 inherently observable** (SAT witnesses proving no fence can fix them), and 6 partial-fence classifications. The 55 UNSAT results are machine-checked certificates; the 40 SAT results prove that patterns like 2+2W, CoWR, CoWW, S, and mp_3thread have forbidden outcomes that persist regardless of fence placement. The complete fence proof catalog appears in Appendix N.

4.5 SMT-Based Litmus Test Generation

We formulate model discrimination as SMT satisfiability over symmetric differences: for each model pair, Z3 identifies patterns satisfiable under one model but not the other. A greedy set-cover identifies a *minimal discriminating set* of 2 patterns (`isa2`, `mp_addr`) covering 5/6 model pairs.

Exhaustive skeleton synthesis independently rediscovers classical litmus tests from first principles:

- **TSO vs. PSO/ARM/RISC-V**: synthesizes the MP pattern.
- **PSO vs. ARM/RISC-V**: synthesizes the LB pattern.
- **ARM vs. RISC-V**: correctly reports no 2-op discriminator.

Details in Appendix P.

4.6 False-Negative Analysis

The 7 non-exact-match cases from the 203-snippet benchmark are formally classified by comparing portability profiles:

Table 4: False-negative classification of 7 near-miss cases.

Category	Count	Safety Consequence
SAFE (conservative)	4	Tool flags extra architectures
NEUTRAL (identical)	3	No safety consequence
UNSAFE (missed bug)	0	—
Total	**7**	**Zero false negatives**

Zero UNSAFE cases: the tool never underreports portability risks on this benchmark. Detailed case-by-case analysis in Appendix O.

4.7 Validation

Against herd7 specifications. We compared all 228 CPU (pattern, architecture) pairs against expected outcomes from herd7ś `.cat` specifications. **Result: 228/228 (100%) agreement** (95% Wilson CI $[98.3\%, 100\%]$). This validates against expected results derived from the specifications, not running herd7 directly; we provide `.litmus` files and a validation script for independent verification.

Hardware validation. 25 predictions validated against published hardware observations from Intel, AMD, ARM Cortex, SiFive RISC-V, and NVIDIA/AMD GPUs. **Result: 25/25 consistency** (3 conservative overapproximations).

Differential testing. 642 meaningful differential tests (monotonicity: 450, fence soundness: 60, custom model cross-validation: 57, litmus round-trip: 75) plus 3,000 determinism checks, all passing with zero mismatches.

Performance. Full 750-pair analysis: <200 ms mean (20 runs, 95% bootstrap CI $[160, 263]$ ms). Individual 2-thread patterns: <1 ms.

5 Related Work

`herd7` [2] simulates memory models from `.cat` specifications with greater generality. LITMUS∞ trades generality for an integrated portability workflow with fence recommendations. Dartagnan [7] performs bounded model checking with explicit portability analysis—the most directly comparable work. Unlike LITMUS∞, Dartagnan handles full programs but does not model GPU scoped synchronization or provide automatic fence recommendations. The tools are complementary: LITMUS∞ for sub-millisecond CI/CD screening, Dartagnan for deep verification. We provide `.litmus` exports for handoff.

Musketeer [3] and VSync [12] perform whole-program fence insertion. MemSynth [6] and Wickerson et al. [18] work at the model-definition level. CDSChecker [11] and GenMC [9] provide C11-specific analysis. GPUVerify [5] checks GPU data races. ThreadSanitizer [16] detects data races dynamically but does not model architecture-specific memory ordering.

Table 5: Feature comparison with related tools.

Feature	**Litmus∞**	**herd7**	**Dart.**	**VSync**	**CDS.**
CPU models	4+DSL	10+	5+	4	1
GPU scope	1 (6 inst.)	0	0	1	0
Portability query	built-in	script	built-in	×	×
Scope mismatch	✓	×	×	partial	×
Per-thread fences	✓	×	×	✓	×
Code analysis	AST	×	full	full	full
Custom models	DSL	.cat	×	×	×
Sub-second	✓	✓	×	×	×
Diff. testing	✓	×	×	×	×

6 Discussion

Scope and limitations. Litmus∞ operates on 75 fixed litmus test patterns, not arbitrary programs. The AST analyzer bridges this gap with 98.0% top-3 accuracy on 203 snippets, but cannot handle complex control flow. Unrecognized patterns trigger explicit warnings with coverage confidence metrics.

Compositional reasoning. Litmus∞ analyzes patterns in isolation and cannot compose results into whole-program guarantees. Two individually safe patterns may interact unsafely when sharing memory locations. Full-program tools (Dartagnan, CDSChecker) are required for compositional reasoning.

GPU model simplification. The 6 GPU configurations are instantiations of a single parameterized model differing only in scope level. The independent model count is 5 (4 CPU + 1 GPU). This captures the scope mismatch phenomenon but not all API-specific subtleties.

Validation methodology. The SMT encodings (both enumerative and Z3-based) were developed by the same authors. The 228/228 + 108/108 agreement demonstrates internal consistency, not external validation. The herd7 comparison and hardware validation provide independent evidence. Mechanization of Theorems 1–3 is future work.

Inherently observable patterns. 40 of 101 unsafe CPU pairs are inherently observable: no fence placement eliminates the forbidden outcome. These arise from coherence patterns (CoWR, CoWW), multi-writer conflicts (2+2W, S), and split-writer patterns (mp_3thread). This classification guides developers when fences are insufficient and architectural redesign (e.g., atomic RMW) is required.

Certificate coverage. The 95 Z3 certificates (55 UNSAT + 40 SAT) cover the 101 unsafe CPU pairs but not the full 750 test-model pair space. Safe pairs are validated by exhaustive enumeration finding no forbidden-outcome-producing execution, cross-checked by the SMT encoding (228/228 + 108/108 agreement) and herd7 (228/228). Extending Z3 certificates to all 750 pairs is tractable but was not pursued as the enumerative and SMT encodings already agree on all pairs.

Benchmark self-generation. The 203-snippet benchmark was assembled by the tool authors, not drawn from an independent corpus with ground-truth labels. The 100% effective safety rate (0/7 false negatives) should be interpreted as absence of evidence for false negatives in this sample, not evidence of absence

in general. Evaluation on an externally curated benchmark (e.g., production concurrent code with known portability bugs) would strengthen external validity.

Encoding fidelity. The tool's model implementations were manually derived from published specifications, not mechanically extracted from official `.cat` files. The 228/228 herd7 agreement provides strong empirical evidence of fidelity for CPU models, but does not constitute a formal proof that the encoding faithfully represents the specifications. The GPU model is an intentionally conservative approximation; vendor-specific subtleties (Vulkan availability/visibility operations, PTX `.weak` modifiers) are not modeled.

7 Conclusion

We presented LITMUS∞, a cross-architecture memory model portability checker that analyzes 75 patterns across 4 CPU models and 1 parameterized GPU model (6 scope instantiations, 750 total pairs) in <200 ms. Key results: 96.6% exact-match accuracy (98.0% top-3) on 203 snippets; 228/228 herd7 agreement; 228/228 CPU + 108/108 GPU SMT internal consistency; 55 UNSAT + 40 SAT Z3 fence certificates; zero false negatives on this benchmark; and 6 critical GPU scope mismatch bugs undetectable by CPU-only tools. 642 meaningful differential tests plus 25/25 hardware consistency provide multi-layered validation.

References

[1] S. V. Adve and H.-J. Boehm. Memory models: A case for rethinking parallel languages and hardware. *Commun. ACM*, 53(12):90–101, 2010.

[2] J. Alglave, L. Maranget, and M. Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM TOPLAS*, 36(2):7:1–7:74, 2014.

[3] J. Alglave, D. Kroening, V. Nimal, and M. Tautschnig. Software verification for weak memory via program transformation. In *Proc. ESOP*, pp. 512–532, 2013.

[4] J. Alglave, M. Batty, A. F. Donaldson, G. Gopalakrishnan, J. Ketema, D. Poetzl, T. Sorensen, and J. Wickerson. GPU concurrency: Weak behaviours and programming assumptions. In *Proc. ASPLOS*, pp. 577–591, 2015.

[5] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson. GPUVerify: A verifier for GPU kernels. In *Proc. OOPSLA*, pp. 113–132, 2012.

[6] J. Bornholt and E. Torlak. Synthesizing memory models from framework sketches and litmus tests. In *Proc. PLDI*, pp. 467–481, 2017.

[7] N. Gavrilenko, H. Ponce de León, F. Furbach, K. Heljanko, and R. Meyer. BMC for weak memory models: Relation analysis for compact SMT encodings. In *Proc. CAV*, pp. 355–373, 2019.

[8] Khronos Group. Vulkan Memory Model. Vulkan Specification 1.2, 2020.

[9] M. Kokologiannakis, O. Lahav, K. Sagonas, and V. Vafeiadis. Effective stateless model checking for C/C++ concurrency. *PACMPL*, 2(POPL):17:1–17:32, 2018.

[10] D. Lustig, S. Sahasrabuddhe, and O. Giroux. A formal analysis of the NVIDIA PTX memory consistency model. In *Proc. ASPLOS*, pp. 257–270, 2019.

[11] B. Norris and B. Demsky. CDSChecker: Checking concurrent data structures written with C/C++ atomics. In *Proc. OOPSLA*, pp. 131–150, 2013.

[12] J. Oberhauser, R. Guerraoui, M. Jaspal, H. Ponce de León, and V. Vafeiadis. VSync: Push-button verification and optimization for synchronization primitives on weak memory models. In *Proc. ASPLOS*, pp. 530–545, 2021.

[13] S. Owens, S. Sarkar, and P. Sewell. A better x86 memory model: x86-TSO. In *Proc. TPHOLs*, pp. 391–407, 2009.

[14] C. Pulte, S. Flur, W. Deacon, J. French, S. Sarkar, and P. Sewell. Simplifying ARM concurrency: Multicopy-atomic axiomatic and operational models for ARMv8. In *Proc. POPL*, pp. 19:1–19:29, 2018.

[15] A. Waterman and K. Asanović, editors. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V Foundation, 2019.

[16] K. Serebryany and T. Iskhodzhanov. ThreadSanitizer – data race detection in practice. In *Proc. WBIA*, pp. 62–71, 2009.

[17] T. Sorensen and A. F. Donaldson. The hitchhiker's guide to cross-platform OpenCL application development. In *Proc. IWOCL*, pp. 2:1–2:12, 2016.

[18] J. Wickerson, M. Batty, T. Sorensen, and G. Constantinides. Automatically comparing memory consistency models. In *Proc. POPL*, pp. 190–204, 2017.

[19] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proc. TACAS*, pp. 337–340, 2008.

[20] ARM Ltd. ARM Cortex-A76 Software Optimization Guide. ARM Documentation, 2019.

[21] A. Fog. Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns. Technical report, Technical University of Denmark, 2023.

[22] SiFive Inc. SiFive U74 Core Complex Manual, v21G1.01, 2021.

A Full Portability Matrix

Table 6: Complete portability matrix (57 patterns × 4 CPU + 6 GPU configurations). ✓ = Safe, × = Unsafe.

	CPU				GPU					
Pattern	**x86**	**SP**	**ARM**	**RV**	**CL-W**	**CL-D**	**VK-W**	**VK-D**	**CTA**	**GPU**
mp	✓	×	×	×	×	×	×	×	×	×
sb	×	×	×	×	×	×	×	×	×	×
lb	✓	✓	×	×	×	×	×	×	×	×
iriw	✓	✓	×	×	×	×	×	×	×	×
2+2w	×	×	×	×	×	×	×	×	×	×
rwc	×	×	×	×	×	×	×	×	×	×
wrc	✓	✓	×	×	×	×	×	×	×	×
mp_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
sb_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
isa2	✓	✓	×	×	×	×	×	×	×	×
r	✓	✓	×	×	×	×	×	×	×	×
mp_data	✓	×	×	×	×	×	×	×	×	×
mp_3thread	×	×	×	×	×	×	×	×	×	×
mp_rfi	✓	×	×	×	×	×	×	×	×	×
sb_3thread	×	×	×	×	×	×	×	×	×	×
lb_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
corr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
cowr	×	×	×	×	×	×	×	×	×	×
coww	×	×	×	×	×	×	×	×	×	×
corw	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
iriw_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
wrc_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
rwc_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
s	×	×	×	×	×	×	×	×	×	×
mp_addr	✓	×	×	×	×	×	×	×	×	×
sb_rfi	×	×	×	×	×	×	×	×	×	×
dekker	×	×	×	×	×	×	×	×	×	×
peterson	×	×	×	×	×	×	×	×	×	×
mp_co	✓	×	×	×	×	×	×	×	×	×
3sb	×	×	×	×	×	×	×	×	×	×
amoswap	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
mp_dmb_st	✓	✓	×	×	×	×	×	×	×	×
mp_dmb_ld	✓	×	×	×	×	×	×	×	×	×
mp_fence_ww_rr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
sb_fence_wr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
lb_fence_rw	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
mp_fence_wr	✓	✓	✓	×	✓	✓	✓	✓	✓	✓
lb_data	✓	✓	✓	✓	×	×	×	×	×	×
wrc_addr	✓	✓	×	×	×	×	×	×	×	×
gpu_mp_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×
gpu_sb_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_sb_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×
gpu_lb_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_wrc_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_iriw_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_iriw_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×
gpu_2plus2w_xwg	×	×	×	×	×	×	×	×	×	×
gpu_rwc_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_3_wg_barrier	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_mp_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_mp_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_sb_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_coher_rr_xwg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_coher_wr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_iriw_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_barrier_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×
gpu_release_acq	✓	✓	✓	✓	×	×	×	×	×	×

B AST Analyzer Benchmark Details

The 203 benchmark snippets span the following real-world systems:

- **Linux kernel** (8): RCU publication, kfifo put/get, spinlock, futex, workqueue, completion wait/complete, RCU dereference
- **Data structures** (9): SPSC queue, Folly MPMCQueue, crossbeam deque, hazard pointers, Tokio MPSC, bounded MPMC, Chase-Lev deque, Treiber stack, Michael-Scott queue
- **Synchronization** (16): Dekker, Peterson, ticket lock, MCS lock, CLH lock, double-checked locking, seqlock, eventcount, Abseil mutex, barrier sense-reversal, RW lock, epoch reclamation, spinlock trylock, flag coordination
- **C++ atomics** (11): relaxed MP/SB/LB, release-acquire MP, consume MP, seq_cst SB/MP, GCC builtins, IRIW release/acquire
- **GPU/CUDA** (9): scope mismatch, device fence, OpenCL WG/global barriers, reduction scope, SB device, WG MP, LB WG
- **RISC-V** (5): fence w,w+r,r, fence w,r pitfall, SB fence w,r, full fence MP/SB

- **Applications** (7): LevelDB SkipList, RocksDB memtable, Folly singleton, database WAL, event loop, channel, connection pool
- **Systems** (3): DPDK ring, virtio descriptor, io_uring SQE
- **Multi-thread** (7): WRC, ISA2, RWC, IRIW, 2+2W variants
- **Other** (11): allocators, coherence, fenced, dependency, basic

Full results are available in the supplementary data.

C Custom Memory Model DSL Specification

C.1 Syntax

Listing 2: Custom model DSL grammar (informal)

```
model <Name> [extends <Parent>] {
    [description "<text>"]
    relaxes <pair> [, <pair>]*
    [preserves deps]
    [not multi-copy-atomic]
    [scope <level>]
    fence <name> [(cost=<N>)] { orders <pair>* }
    ...
}
<pair> ::= W->R | W->W | R->R | R->W
```

C.2 Built-in Example Models

- **SC**: Sequential consistency (no relaxation)
- **POWER**: IBM POWER (all relaxed, deps preserved, not MCA; hwsync, lwsync, isync fences)
- **Alpha**: DEC Alpha (all relaxed, no deps, not MCA; mb, wmb, rmb fences)
- **C11-relaxed**: C11 with relaxed atomics (all relaxed, deps preserved, not MCA; seq_cst, release, acquire, acq_rel fences)

C.3 DSL vs. Built-in Model Expressiveness

Cross-validation between DSL-defined TSO and built-in x86/TSO achieves **57/57 (100%) agreement** on all CPU patterns. The DSL checker uses the same exhaustive execution enumeration as the built-in engine, properly modeling coherence ordering, cross-thread communication effects, and fence semantics. This full model checking approach eliminates a prior expressiveness gap where an earlier version of the DSL used only intra-thread reordering analysis, which missed coherence effects in patterns like `cowr`, `coww`, and `s`.

Root cause analysis of prior mismatches. The 10 mismatches present in an earlier version arose from two bugs: (1) a regex parsing error that failed to capture fence ordering specifications from inline DSL definitions, and (2) the DSL checker used a simplified per-thread reordering analysis instead of the full $RF \times CO$ enumeration. Both issues are now resolved: the DSL parser correctly extracts fence specifications, and the checker delegates to the same verification algorithm as the built-in engine (Algorithm 1).

D Hardware Validation Details

All 25 hardware validation points:

- **x86** (7): MP, SB, LB, IRIW, 2+2W, Dekker, CoRR — Owens et al. [13], Alglave et al. [2]
- **ARM** (8): MP, SB, LB, IRIW, WRC, MP+fence, SB+fence, CoRR — Pulte et al. [14], Alglave et al. [2]
- **SPARC** (3): MP, SB, LB — Alglave et al. [2]

- **RISC-V** (3): MP, SB, LB — RISC-V ISA specification litmus tests
- **GPU** (4): gpu_mp_wg, gpu_mp_dev, gpu_barrier (OpenCL, PTX) — Sorensen & Donaldson [17], Alglave et al. [4]

In all 25 cases, the tool's prediction is consistent with published hardware behavior. 3 cases show conservative overapproximation (tool predicted unsafe, outcome not observed on tested hardware).

E Differential Testing Framework

The differential testing framework performs five categories of automated cross-validation:

1. **Monotonicity** (450 checks): For each pair (M_1, M_2) where $M_1 \sqsubseteq M_2$, verify that if pattern T is safe on M_2 then it is safe on M_1. Checked pairs: TSO/PSO, PSO/ARM, PSO/RISC-V, and GPU Dev/WG per API.
2. **Fence soundness** (60 checks): For 6 unfenced/fenced pattern pairs (mp/mp_fence, sb/sb_fence, etc.), verify that the fenced version is never unsafe when the unfenced is safe.
3. **Determinism** (3,000 checks): Run the full 750-pair analysis 5 times and verify identical results.
4. **Custom model cross-validation** (57 checks): Compare DSL-defined TSO against built-in x86/TSO using the same exhaustive verification algorithm. All 57 checks pass with zero mismatches.
5. **Litmus file round-trip** (75 checks): Export all patterns as `.litmus` files, verify each has valid structure (header, init block, exists clause).

F RISC-V Asymmetric Fence Analysis

RISC-V provides `fence pred,succ` where pred/succ $\in$ {r, w, rw}. The tool includes 6 asymmetric fence patterns. The sole ARM/RISC-V discriminator (`mp_fence_wr`) uses `fence w,r` on the producer—which provides W→R ordering but *not* the W→W ordering required for the MP producer thread's two stores to be seen in order.

The AST analyzer now correctly identifies RISC-V asymmetric fences with `fence_pred/fence_succ` specificity matching (weight 3 in similarity), achieving 100% accuracy on 5 RISC-V benchmark snippets.

G Fence Cost Analysis Details

Cost model.

- ARM: `dmb ishst` (W→W, cost 1), `dmb ishld` (R→R and R→W, cost 2), `dmb ish` (full, cost 4).
- RISC-V: `fence r,r` (cost 1), `fence w,w` (cost 1), `fence r,w` (cost 1), `fence w,r` (cost 2), `fence rw,rw` (full, cost 4).

Aggregate statistics. 55 pattern-architecture pairs show fence savings. ARM: mean 50.9% ± 32.4% (median 62.5%, 95% bootstrap CI [38.6%, 62.7%]). RISC-V: mean 66.5% ± 26.8% (median 75.0%, 95% bootstrap CI [56.3%, 75.8%]). The high variance reflects the diversity of patterns: simple 2-thread patterns achieve >75% savings while complex multi-thread patterns (e.g., `iriw` requiring full barriers) achieve lower savings.

Table 7: Per-thread fence cost savings (representative patterns). Costs are analytical weights reflecting ordering strength, not hardware latencies.

Pattern	Arch	Minimal Fences	Savings
iriw	ARM	`dmb ishld` (T2, T3)	75.0%
mp	ARM	`ishst` (T0); `ishld` (T1)	62.5%
iriw	RISC-V	`fence r,r` (T2, T3)	87.5%
mp	RISC-V	`fence w,w` (T0); `r,r` (T1)	75.0%

Important: These are *ordering strength reductions*, not hardware performance improvements. Actual throughput impact depends on microarchitectural state, workload, and contention.

Table 8: Published fence instruction latencies (cycles). Values are representative ranges from architecture manuals, not our own measurements.

Architecture	Fence	Cycles	Source
ARM Cortex-A76	`dmb ish`	30–70	ARM optimization guide [20]
ARM Cortex-A76	`dmb ishst`	10–30	ARM optimization guide [20]
ARM Cortex-A76	`dmb ishld`	8–25	ARM optimization guide [20]
x86 (Skylake)	`MFENCE`	33–100	Agner Fog tables [21]
x86 (Skylake)	`LFENCE`	4–5	Agner Fog tables [21]
RISC-V (SiFive U74)	`fence rw,rw`	20–60	SiFive manual [22]
RISC-V (SiFive U74)	`fence w,w`	10–25	SiFive manual [22]
RISC-V (SiFive U74)	`fence r,r`	5–15	SiFive manual [22]

H herd7 Litmus File Format

All 57 patterns are exported as standard C-language `.litmus` files in the `litmus_files/` directory. Each file follows herd7's expected format with initial state, per-thread code using `WRITE_ONCE`/`READ_ONCE`, and an `exists` clause specifying the forbidden outcome.

To validate with herd7:

```
# Install herd7: opam install herd7
# Run validation:
bash validate_herd7.sh
# Or manually:
herd7 -model aarch64.cat litmus_files/mp.litmus
```

I Performance Details

Full 750-pair analysis: mean <200 ms (20 runs). Individual 2-thread patterns: <1 ms. Most expensive pattern (2+2W, 3 threads): ∼6 ms.

AST code analysis: <5 ms per snippet on average (203 snippets). The tree-sitter parse step is <1 ms; pattern matching dominates.

J Reproducibility

Listing 3: Reproducing all experiments

```
# Full 750-pair portability analysis
python3 portcheck.py --analyze-all

# AST-based code analyzer benchmark (203 snippets)
python3 benchmark_suite.py

# Custom model experiments
python3 model_dsl.py

# Differential testing (3,642 checks)
python3 differential_testing.py

```

```
# SMT-based formal validation (Z3)
python3 smt_validation.py

# herd7 validation
python3 herd7_validation.py

# Generate all paper results
python3 run_paper_experiments.py
```

All data files are provided as supplementary material and can be verified against the numbers in this paper.

K Category Safety Rate Analysis

Table 9: Safety rates by pattern category.

Category	Patterns	Safe/Total	Rate
Fenced	10	99/100	99.0%
Asymmetric fence	6	42/60	70.0%
GPU scope	18	125/180	69.4%
Coherence	4	20/40	50.0%
Multi-thread	22	86/220	39.1%
Dependency	6	12/60	20.0%
Basic ordering	9	9/90	10.0%
Mutex	2	0/20	0.0%

L Addressing Known Limitations

Regarding Dartagnan comparison. Dartagnan [7] is the most directly comparable tool, supporting explicit portability analysis between memory models with full program analysis. A head-to-head comparison on the same litmus tests would be informative. We provide `.litmus` exports and encourage users to run Dartagnan on these files. Key differences: Dartagnan uses SAT/SMT-based bounded model checking on full C programs, while Litmus∞ uses brute-force enumeration on litmus patterns. Dartagnan handles larger programs but does not model GPU scoped synchronization.

Regarding code analysis scope. Our AST-based code analyzer is a *pattern matcher*, not a full program analyzer. It maps code snippets to the nearest litmus test pattern from a fixed database. This is by design: the tool provides fast, actionable diagnostics ("your code matches the MP pattern, which is unsafe on ARM") rather than full-program verification. For full program analysis, we recommend using Litmus∞ in conjunction with Dartagnan or VSync.

Regarding GPU model fidelity. Our 6 GPU configurations (3 APIs $\times$ 2 scope levels) are instantiations of a single parameterized model differing only in scope-level identifiers. The independent model count is 5 (4 CPU + 1 parameterized GPU), not 10. This is intentionally conservative: it captures the *scope mismatch* phenomenon (the main contribution) without modeling API-specific subtleties (Vulkan availability/visibility, PTX `.weak` modifiers). More precise GPU models would reduce false positives.

M SMT Encoding Details

The SMT encoding translates each (pattern, model) pair into a Z3 formula. We describe the encoding for a litmus test T with threads $t_1, \ldots, t_n$, addresses $a_1, \ldots, a_m$, and model M.

Variables.

- $rf_\ell \in \{0, \ldots, k_\ell - 1\}$ for each load ℓ: which store (including initial store) the load reads from. k_ℓ is the number of stores to ℓ's address.
- $co_{a,i,j} \in \{true, false\}$ for non-initial stores i, j to address a: whether i precedes j in coherence order.
- $ts_e \in \mathbb{Z}_{\geq 0}$ for each event e: a timestamp encoding the topological ordering (edges imply $<$).

Constraints.

1. **Coherence totality**: for each address a and distinct non-initial stores i, j: $co_{a,i,j} \oplus co_{a,j,i}$ and transitivity.
2. **Model-dependent po**: for each in-order pair (u, v) on the same thread, if the model preserves this ordering, assert $ts_u < ts_v$.
3. **rf edges**: $rf_\ell = k \implies ts_{s_k} < ts_\ell$ where s_k is the k-th store.
4. **co edges**: $co_{a,i,j} \implies ts_i < ts_j$.
5. **fr edges**: $rf_\ell = k \wedge co_{a,k,j} \implies ts_\ell < ts_j$.
6. **Forbidden outcome**: conjunction of $rf_\ell \in \{k : val(s_k) = F_T[\ell.reg]\}$ for each load in the forbidden set.

Soundness argument. The SMT formula is satisfiable iff there exists a consistent execution producing the forbidden outcome. The timestamp encoding faithfully represents acyclicity: a total order on timestamps exists iff the edge graph is acyclic. This encoding is equivalent to the enumeration-based checker, as confirmed by the 228/228 agreement.

Fence sufficiency proofs. For each unfenced/fenced pair (T, T_f), we check: (1) $\varphi(T, M)$ is SAT (unfenced is unsafe), and (2) $\varphi(T_f, M)$ is UNSAT (fenced is safe). The UNSAT result constitutes a machine-checked proof that the fence eliminates all consistent executions producing the forbidden outcome. All 55 fixable pairs verified with UNSAT fence proofs. An additional 40 inherently observable pairs are proven unfixable via SAT results on both unfenced and fenced versions.

N Comprehensive Fence Proof Catalog

Table 10 lists all 101 unsafe CPU pairs with their SMT classification. Each "fence-sufficient" entry has a machine-checked UNSAT certificate proving that the recommended fence eliminates the forbidden outcome. Each "inherently observable" entry has SAT witnesses proving that the forbidden outcome persists regardless of fence placement.

Table 10: Complete fence sufficiency classification (101 unsafe CPU pairs).

Pattern	**Model**	**Category**	**SMT Result**
3sb	TSO	fence-sufficient	unf=sat, fen=unsat
3sb	PSO	fence-sufficient	unf=sat, fen=unsat
3sb	ARM	fence-sufficient	unf=sat, fen=unsat
3sb	RISC-V	fence-sufficient	unf=sat, fen=unsat
dekker	TSO	fence-sufficient	unf=sat, fen=unsat
dekker	PSO	fence-sufficient	unf=sat, fen=unsat
dekker	ARM	fence-sufficient	unf=sat, fen=unsat
dekker	RISC-V	fence-sufficient	unf=sat, fen=unsat
iriw	ARM	fence-sufficient	unf=sat, fen=unsat
iriw	RISC-V	fence-sufficient	unf=sat, fen=unsat
isa2	ARM	fence-sufficient	unf=sat, fen=unsat
isa2	RISC-V	fence-sufficient	unf=sat, fen=unsat
lb	ARM	fence-sufficient	unf=sat, fen=unsat
lb	RISC-V	fence-sufficient	unf=sat, fen=unsat
mp	PSO	fence-sufficient	unf=sat, fen=unsat

Pattern	Model	Category	SMT Result
mp	ARM	fence-sufficient	unf=sat, fen=unsat
mp	RISC-V	fence-sufficient	unf=sat, fen=unsat
mp_addr	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
mp_co	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
mp_data	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
mp_rfi	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
peterson	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
r	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
rwc	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
sb	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
sb_3thread	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
sb_rfi	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
wrc	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
wrc_addr	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
2+2w	all 4 CPU	inherent	unf=sat, fen=sat
cowr	all 4 CPU	inherent	unf=sat, fen=sat
coww	all 4 CPU	inherent	unf=sat, fen=sat
mp_3thread	all 4 CPU	inherent	unf=sat, fen=sat
s	all 4 CPU	inherent	unf=sat, fen=sat
mp_dmb_ld	PSO/ARM/RV	partial	fix: mp_fence
mp_dmb_st	ARM/RV	partial	fix: mp_fence
mp_fence_wr	RISC-V	partial	fix: mp_fence

Why inherently observable patterns are unfixable. The 5 base patterns (2+2W, CoWR, CoWW, S, mp_3thread) have forbidden outcomes arising from fundamental properties of concurrent memory systems:

- **2+2W**: Two threads write two addresses in opposite order. The observer can see inconsistent coherence orders across addresses because fences order operations within a thread but cannot constrain cross-address coherence order as seen by a third thread.
- **CoWR, CoWW**: Same-address coherence patterns. The forbidden outcome arises from the non-deterministic interleaving of cross-thread writes to the same address, which fences cannot prevent.
- **S**: Two threads write to the same address; the forbidden outcome depends on which writer is coherence-last, which is unconstrained by intra-thread fences.
- **mp_3thread**: The writes that must be ordered are on *different* threads, so no intra-thread fence can order them.

O False-Negative Analysis Details

Table 11 shows the full classification of all 7 non-exact-match cases from the 203-snippet benchmark. These are snippets where the exact-match prediction differs from the expected pattern.

Table 11: Detailed false-negative analysis of 7 non-exact-match cases.

Snippet	Expected→Predicted	Class	Reason
lb_data_dep	lb_data→lb	SAFE	lb flags ARM, RISC-V (extra)
linux_rcu_deref	mp_addr→mp	NEUTRAL	identical profile
linux_smp_wmb_rmb	mp_fence→mp	SAFE	mp flags SPARC, ARM, RISC-V (extra)
dekker_mutual_excl	dekker→sb	NEUTRAL	identical profile

Snippet	Expected→Predicted	Class	Reason
cpp_seq_cst_mp	mp_fence→mp	SAFE	mp flags SPARC, ARM, RISC-V (extra)
gcc_builtin_rel_acq	mp_fence→mp	SAFE	mp flags SPARC, ARM, RISC-V (extra)
mp_address_dep	mp_addr→mp	NEUTRAL	identical profile

All 4 SAFE cases involve the predicted pattern being *more* conservative: it flags a superset of the architectures the expected pattern flags. The 3 NEUTRAL cases have portability profiles identical to the expected pattern. Zero UNSAFE cases: the tool never underreports portability risks.

P SMT-Based Model Discriminator Details

The SMT-based litmus test generation identifies patterns in the symmetric difference of two models' allowed execution sets. For each model pair (M_A, M_B), a pattern T is *discriminating* if $\varphi(T, M_A)$ is SAT while $\varphi(T, M_B)$ is UNSAT (or vice versa).

The minimal discriminating set {`isa2`, `mp_addr`} covers 5 of 6 model pairs:

- `isa2` discriminates TSO/PSO from ARM/RISC-V (dependency-based patterns safe under TSO/PSO due to preserved ordering, unsafe under ARM/RISC-V).
- `mp_addr` discriminates TSO from PSO (address-dependency pattern safe under TSO, unsafe under PSO which relaxes store-store order).

The ARM vs. RISC-V pair cannot be discriminated by any base (unfenced) pattern, as confirmed by 0 discriminating tests. The sole discriminator is the asymmetric fence test `mp_fence_wr`, which is safe on ARM (both fences effective) but unsafe on RISC-V (the `fence w,r` on the producer does not order the W→W pair).

Q Mismatch Root Cause Analysis

Table 12: Root cause taxonomy of the 10 original DSL cross-validation mismatches.

Pattern	Root Cause	Category	Severity
mp_fence	Regex bug	Implementation	High
sb_fence	Regex bug	Implementation	High
lb_fence	Regex bug	Implementation	High
iriw_fence	Regex bug	Implementation	High
wrc_fence	Regex bug	Implementation	High
rwc_fence	Regex bug	Implementation	High
mp_fence_ww_rr	Simplified analysis	Algorithmic	Medium
sb_fence_wr	Simplified analysis	Algorithmic	Medium
lb_fence_rw	Simplified analysis	Algorithmic	Medium
mp_fence_wr	Simplified analysis	Algorithmic	Medium

The 10 original mismatches (version 3).

Chi-squared test. H_0: mismatches are uniformly distributed across pattern-model pairs.
H_1: mismatches are systematically concentrated.
Result: $\chi^2 = 184.2$, $df = 27$, $p < 0.001$. We reject H_0: mismatches are highly systematic. Note that expected cell frequencies are below 5 due to the small number of mismatches (10), so this result should be interpreted as directional evidence rather than an exact test.

McNemar's test (post-fix). Comparing DSL-defined TSO vs. built-in x86/TSO on 57 CPU patterns: Discordant: 0 (in both directions).
With zero discordant observations, McNemar's test has no power; the relevant observation is simply that **0/57 disagreements** were found.

R Pattern Completeness Analysis

Architecture boundary coverage. Our 57 patterns cover all 9 architecture model boundaries:

Table 13: Architecture boundary discrimination coverage.

Boundary	Discriminators	Coverage
TSO → PSO	6	100%
PSO → ARM	7	100%
PSO → RISC-V	8	100%
ARM → RISC-V	1	100%
GPU Dev → WG (×3)	5 each	100%
TSO → ARM	13	100%
TSO → RISC-V	14	100%

Information-theoretic analysis. The 57 CPU patterns produce 5 unique safety fingerprints (binary vectors over 4 architectures). The joint entropy of the fingerprint distribution is 1.85 bits out of a maximum of $\log_2(5) = 2.32$ bits, indicating moderate concentration: most patterns cluster into a few fingerprint classes. A greedy set-cover algorithm identifies 3 patterns (`iriw`, `mp`, `mp_fence_wr`) as a minimal discriminating set—sufficient to distinguish all CPU architecture boundaries. The redundancy is deliberate: multiple patterns per boundary increase confidence and cover edge cases (dependency variants, multi-thread configurations) that a minimal set would miss.

S Algorithm Details

Algorithm 1 Cross-Architecture Portability Check

Require: Pattern T, target model M
Ensure: Safe/Unsafe, fence recommendation

```
Let RF = all reads-from assignments for T
Let CO = all coherence orders for T
for each (rf, co) ∈ RF × CO do
    ghb ← ppo(M) ∪ rf_e ∪ co ∪ fr
    if ghb acyclic and outcome = F_T then
        return (Unsafe, MinFences(T, M))
end for
return (Safe, ⊥)
```

The consistency check incorporates: (1) preserved program order (model-dependent), (2) fence effectiveness including GPU scope, (3) dependency preservation (ARM/RISC-V), (4) multi-copy atomicity, and (5) RISC-V asymmetric fence pred/succ sets.

Algorithm 2 Per-Thread Fence Selection

Require: Pattern T, model M, fence menu $\mathcal{F}_M$
Ensure: Per-thread fence map $\phi : \text{threads}(T) \to \mathcal{F}_M$

```
for each thread t in T do
    V_t ← {(b, a) : b --po--> a on t, b.addr ≠ a.addr, (b.type, a.type) relaxed by M, not covered by existing fences or dependencies}
    if V_t = ∅ then
        φ(t) ← ⊥ {no fence needed}
    else
        φ(t) ← arg min_{f∈F_M} cost(f) s.t. V_t ⊆ covers(f)
    end if
end for
return φ
```

T Proofs

Lemma 4 (RF×CO Decomposition)**.** *For a litmus test T with n loads, m stores per address, and k addresses, the set of candidate executions is precisely $RF \times CO$ where $|RF| \leq \prod_{\ell=1}^{n}(m_\ell + 1)$ (each load reads from one of the stores to the same address or the initial value) and $|CO| = \prod_{a=1}^{k} m_a!$ (each per-address coherence order is a permutation of the stores to that address). Every consistent execution under any model M that respects coherence and reads-from well-formedness is a member of this product set.*

Proof of Lemma 4. By the axiomatic framework [2], an execution is fully determined by (1) the program order (fixed by T), (2) the reads-from function rf, and (3) the coherence order co. Reads-from maps each load to a store on the same address or the initial write; coherence orders stores per-address. The Cartesian product $RF \times CO$ therefore contains all candidate executions. The tool iterates over all (rf, co) pairs: for each, it constructs the ghb relation and checks acyclicity. Since the iteration is exhaustive over this finite product set, no candidate execution is missed. □

Proof of Theorem 1. The proof assumes that the tool's ghb acyclicity check correctly implements the model predicate M_B^{tool} (empirically supported by the 228/228 herd7 agreement, but not formally verified). By Lemma 4, the enumeration covers all elements of $RF \times CO$, the complete set of candidate executions for the finite litmus test. For each candidate, the ghb acyclicity check determines consistency under M_B^{tool}. Since the enumeration is exhaustive, if no consistent execution produces F_T, then F_T is genuinely forbidden under M_B^{tool}. By the permissiveness assumption, every hardware-allowed execution is also tool-allowed, so F_T is also forbidden under M_B^{hw}. The tool may report false positives (safe code flagged as unsafe) when M_B^{tool} is strictly more permissive, but never false negatives. □

Proof of Theorem 2. The algorithm selects $\phi(t) = \arg\min_f cost(f)$ subject to $V_t \subseteq covers(f)$. For any fence f' with $cost(f') < cost(\phi(t))$, we must have $V_t \not\subseteq covers(f')$ (otherwise $\phi(t)$ would not be the minimum). Thus $\phi(t)$ is the minimum-cost fence in $\mathcal{F}_M$ that covers all violated pairs on thread t. □

Remark 2 (Scope and triviality of minimality)**.** *Theorem 2 is a direct consequence of the* arg min *construction in Algorithm 2: the claim is trivially true by definition of the algorithm. Its value is not as a deep mathematical result, but as a precise statement of the guarantee the tool provides—per-thread, per-fence-menu minimality.*

The non-trivial *version of this problem—whole-program fence minimality—asks: given a concurrent program and a target memory model, find the minimum-cost set of fences at arbitrary program points such that all forbidden outcomes are eliminated. This is NP-hard in general by reduction from minimum vertex multicut [3]; tools like Musketeer [3] and VSync [12] address this via ILP or heuristic relaxations.* Litmus∞ *deliberately sidesteps this complexity by operating on litmus-test-sized patterns where per-thread fence selection is tractable and deterministic.*

Proof of Theorem 3. Under model M_S, the preserved program order *ppo* includes fence-mediated edges $a \xrightarrow{\text{po}} f \xrightarrow{\text{po}} b$ only when the fence is effective at scope S.

(*Only if*): A workgroup-scoped fence is effective only within the workgroup: if $t_a \in W_i$ and $t_b \in W_j$ with $i \neq j$, the device-scope model requires device-level visibility, which a workgroup-scoped fence cannot provide. The edge (a, b) is therefore excluded from *ppo*. If this fence edge was the sole ordering source for (a, b), the ghb relation may become acyclic for the forbidden outcome, allowing the bug.

(*If*): If $\sigma_f \succeq S$, the fence provides ordering at scope S or wider, so the edge is included in *ppo*. If the communicating threads share a workgroup, a workgroup-scoped fence suffices because intra-workgroup visibility is sufficient for the model.

This characterizes when the fence *edge* is included in *ppo*, not when the overall forbidden outcome is eliminated. Ordering may also arise from dependencies and release-acquire pairs through separate *ppo* edges not governed by this theorem. □

Remark 3 (Mechanization status)**.** *Theorems 1–3 are paper proofs, not mechanized in a proof assistant (Coq, Isabelle/HOL, or Lean). However, the tool provides two layers of machine-checked evidence: (1) the 228/228 SMT internal consistency constitutes a Z3-verified soundness witness for CPU models: an independent encoding confirms that the enumeration-based checker agrees with Z3 on every CPU (pattern, model) pair; (2) the 55 fence-sufficient UNSAT certificates and 40 inherent-observability SAT witnesses are Z3-generated proof artifacts. Full mechanization remains future work; Section 6 discusses this limitation.*

U GPU Scope Mismatch Details

Table 14: GPU scope mismatch bugs: 6 critical + 5 warning patterns.

Pattern	**Failure Mode**	**Sev.**
gpu_mp_scope_mismatch	All GPU fail	Crit.
gpu_sb_scope_mismatch	All GPU fail	Crit.
gpu_iriw_scope_mm	All GPU fail	Crit.
gpu_barrier_scope_mm	All GPU fail	Crit.
gpu_release_acquire	All GPU fail	Crit.
lb_data	All GPU fail	Crit.
gpu_sb_dev	WG-only fail	Warn.
gpu_wrc_dev	WG-only fail	Warn.
gpu_iriw_dev	WG-only fail	Warn.
gpu_rwc_dev	WG-only fail	Warn.
gpu_mp_dev	WG-only fail	Warn.

The 6 critical bugs represent fundamental CPU/GPU model differences. `lb_data` is safe on all CPUs (data dependency preservation) but fails on all GPUs (no dependency preservation in our GPU models). The 5 warnings are scope-level discriminators: safe with device scope, unsafe with workgroup scope only.

V AST Code Analysis Details

Phase 1: Operation extraction. Tree-sitter produces a concrete syntax tree from which we extract:

- **Atomic operations**: `std::atomic` stores/loads, GCC builtins (`__atomic_store_n`), C11 `atomic_store`, and plain assignments to shared variables.
- **Memory ordering**: `memory_order_relaxed` through `seq_cst`, enabling ordering-aware pattern matching.
- **Fences**: `atomic_thread_fence`, `__sync_synchronize`, ARM `dmb` variants, RISC-V `fence pred,succ` (with asymmetric fence specificity matching), CUDA `__threadfence_block`/`__threadfence`, OpenCL barriers.

- **GPU scope and workgroup**: CTA vs. device scope inferred from `__threadfence_block` vs. `__threadfence`, cross-workgroup communication detected from comments and block indices.

Phase 2: Dependency inference. For each thread, we track register-to-variable bindings and infer data, address, and control dependencies.

Phase 3: Structural pattern matching. Each extracted operation sequence is matched against all 57 built-in patterns using a weighted similarity metric over 12 features: thread count (weight 3), operation counts (weight 2), per-thread sequences (weight 4), cross-thread communication (weight 3), address count (weight 1.5), fence presence (weight 1.5), scope match (weight 1.5), RISC-V fence pred/succ specificity (weight 3), GPU scope type (weight 2.5), scope mismatch (weight 2), dependency types (weight 1), and cross-workgroup (weight 1).

Phase 4: Coverage confidence and unrecognized pattern warnings. Because the tool operates on a fixed pattern database, code that does not match any known pattern would silently produce no output—a dangerous blind spot where developers may interpret silence as safety confirmation. To address this, the analyzer computes a `coverage_confidence` metric: the ratio of concurrent operations explained by the best-matching pattern to the total concurrent operations detected. When `coverage_confidence` < 0.5, the tool emits an `UnrecognizedPatternWarning` listing the unrecognized operations, alerting developers that the analysis is incomplete and manual review or a more powerful tool (e.g., Dartagnan) is needed.

W Custom Model and Boundary Analysis

Table 15: Custom model portability (57 CPU patterns).

Model	Safe	Unsafe	Notes
SC	47	10	10 coherence/multi-thread patterns unsafe even under SC
POWER	27	30	Similar to ARM (all reorderings relaxed)
Alpha	27	30	No dependency preservation
C11-relaxed	27	30	Like ARM without MCA

Table 16: Model boundary discriminators.

Boundary	N	Key Discriminators
TSO → PSO	6	mp, mp_data, mp_rfi, mp_addr, mp_co, mp_dmb_ld
PSO → ARM	7	lb, iriw, wrc, isa2, r, wrc_addr, + 1
ARM → RISC-V	1	mp_fence_wr (sole discriminator)
GPU-Dev → WG	5	gpu_sb/wrc/iriw/rwc/mp_dev

ARM→RISC-V: The sole discriminator `mp_fence_wr` uses asymmetric `fence w,r`. On ARM, symmetric `dmb` covers all orderings. On RISC-V, `fence w,r` provides W→R ordering but not the W→W ordering needed for the MP producer.

X Tool Comparison Details

Y Statistical Power Analysis

Post-hoc power analysis for the 203-snippet benchmark:

Table 17: LITMUS∞ vs. Dartagnan: complementary approaches to memory model portability.

Dimension	**Dartagnan**	**LITMUS∞**
Approach	Full-program BMC	Pattern enumeration
Speed	Seconds–minutes	<1 ms/pattern
Input scope	Whole programs	75 litmus patterns
CPU models	8+ (incl. LKMM, C11)	4 + custom DSL
GPU support	No	Yes (scoped sync.)
Fence recs.	No	Per-thread minimal
CI/CD ready	Slow	Built-in Action
Guarantees	Bounded sound	RF×CO complete

- **Top-3 accuracy**: 199/203 (98.0%), Wilson CI $[95.0\%, 99.2\%]$. Power to detect true rate $< 90\%$ is $> 99\%$.
- **Exact-match accuracy**: 196/203 (96.6%), Wilson CI $[93.1\%, 98.3\%]$, Clopper-Pearson CI $[93.0\%, 98.6\%]$. With $n = 203$, the margin of error is $\pm 2.6\%$ (below the $\pm 5\%$ target).

The Clopper-Pearson intervals confirm Wilson intervals are not anti-conservative.